1455A083《基础实践》(2) 期末报告

2026~2027 秋季学期

学号 24122114　　姓名 韦炳炎　　得分_____

一、实践总结

这次作业中，我先围绕 Text-to-SFX 做了应用调研，再以《原神》稻妻镇守之森为对象制作环境声。最终完成十二个正式 WAV、REAPER 工程、Wwise 工程和九十秒试听。声音方向确定为哀怨轻柔，以持续的风声、虫鸣和远处流水为底色，穿插少量鸟鸣、草丛和铃音。

对我来说，收获最大的是学会根据实际听感取舍。第一批风声听起来太杂，像雨声；重新生成后，我又提出风更轻、叶声更少，继续选段、滤波和降低音量。最终混音比一开始更接近场景需要，也让我认识到“生成出来”和“适合使用”之间还有一段编辑过程。

通过这次实践，我把文件规格、循环、随机容器和空间衰减联系起来。48 kHz、24 bit、双声道解决资产规格问题；循环与随机等待控制环境的连续性和疏密；距离衰减形成远近关系。制作使用了 AI 辅助生成、处理和工程配置，我负责场景方向与试听判断，具体参数和测试记录保留在工程中。

作品与调研入口

课程仓库

https://github.com/kirito-firefly/SHU-Game-Audio-Practice

平时作业

https://github.com/kirito-firefly/SHU-Game-Audio-Practice/blob/main/Task/AIGC_Game_Audio_Research.md

期末作品

https://github.com/kirito-firefly/SHU-Game-Audio-Practice/tree/main/Final

二、平时作业

1. GitHub 账户创建情况

GitHub 用户名为 kirito-firefly，已建立公开课程仓库，用于保存平时调研与期末工程。仓库通过 README 提供入口，期末文件按声音资产、剪辑工程、Wwise 工程、源素材、试听和测试证据分类；缓存、备份和安装文件不上传。

账户主页：https://github.com/kirito-firefly

课程仓库：https://github.com/kirito-firefly/SHU-Game-Audio-Practice

2. “当下 AIGC 在游戏音频设计中的应用”调研

Text-to-SFX 在游戏音效设计中的应用调研

完整调研链接：https://github.com/kirito-firefly/SHU-Game-Audio-Practice/blob/main/Task/AIGC_Game_Audio_Research.md

调研时间：2026 年 10 月；资料复核：2026 年 11 月 12 日。 本文为资料调研。除期末作品实际使用的 Stable Audio 外，其他工具的能力来自官方说明，未做同条件音质对比测试。

1. 从“找音效”到“生成音效”

传统游戏音效制作通常依赖三种方式：

录音、购买/搜索音效库、后期声音设计。

Text-to-SFX 提供了第四种方式：

直接用自然语言描述需要的声音，由生成模型制作音频素材。

例如：

Heavy metal sword hits an ice shield, sharp impact followed by cracking ice.

模型可以直接生成“金属剑撞击冰盾并产生碎裂”的声音。

但截至 2026 年，真正值得关注的已经不只是“文字能不能生成声音”，而是：

能否精确控制声音、能否修改已有声音、能否直接进入专业制作流程、能否理解一个完整游戏场景中的声音关系。

2. 2026 年几个值得关注的方向

ElevenLabs Sound Effects V2：实用型 Text-to-SFX

ElevenLabs 的 Sound Effects V2 已经提供了较完整的文字音效生成流程。

它可以控制：

音效持续时间、Prompt 遵循程度、无缝循环、多版本生成、API 自动化调用。

这对于游戏尤其有价值。

例如森林风声、机器运转、雨声可以直接生成循环版本；攻击、脚步和 UI 声音则可以批量生成多个变体，再交给 Wwise / FMOD 随机播放。

它代表的是当前比较成熟的：

Prompt → 游戏声音资产

工作流。

11 月 12 日复核时，官方已于 10 月 29 日发布 SFX v3。官方介绍其改进了多指令提示词的遵循和部分高频声音的自然度；这些是厂商说明，不能直接当作本次作业的实测排名。当前文档也将 V2 标为弃用，计划于 2027 年 3 月 1 日移除。因此，V2 适合作为原调研中的工作流案例，后续选型应同时核对 V3、接口与条款。[1][2]

Adobe Firefly：从 Text-to-SFX 转向“表演控制”

Adobe Firefly 的官方网页和移动端文档已经介绍了 Generate Sound Effects 的使用流程。[3][4]

它比较特别的地方是 Voice to Sound Effects。

设计者不仅可以输入文字，还可以自己用嘴模仿声音的节奏和强弱：

“唰——唰——砰！”

AI 再根据这段表演生成真正的挥剑与撞击音效。

这解决了纯文字生成的一个明显问题：

文字很容易描述“是什么声音”，却很难描述“什么时候发生、力度如何变化”。

对于攻击动画、角色动作、过场演出等需要严格对齐时间的游戏声音，这种控制方式比单纯写 Prompt 更接近真正的声音设计。

Stable Audio 3.0：生成开始进入 DAW

Stability AI 在 2026 年推出 Stable Audio 3.0。

除了 Text-to-Audio，它还支持：

Sound Effects、Audio-to-Audio、Inpainting、Continuation、对已有音频进行局部修改。

Stable Audio 也提供了 DAW 插件。需要注意，官方插件指南列出的首发平台是 macOS VST 与 AU；Windows 支持仍列在后续更新中。因此，本次 Windows 电脑采用网页生成，再导入 REAPER，没有使用该插件。[5][6]

这意味着生成式 AI 不再只是：

“打开网站 → 生成 → 下载 WAV”

而是开始直接进入传统数字音频工作站。

未来更可能出现：

录音 → AI 修改 → 人工编辑 → AI 补全 → 混音

这样的混合工作流。

Seed Audio 1.0：从“音效”走向“声音场景”

字节跳动 Seed 团队在 2026 年 7 月发布 Seed Audio 1.0。

它代表了另一个更激进的方向：

不再分别生成：

配音、环境声、脚步、音效。

而是直接描述一个完整场景。

例如：

夜晚的废弃工厂，一名角色小声说话，远处机器持续运转，随后右侧传来金属物掉落的声音。

模型尝试在同一个生成过程中同时处理：

Speech + SFX + Ambience + Timing + Acoustic Space

也就是说，AI 的目标正在从：

生成一个声音文件

逐渐变成：

理解并生成一个完整声音场景。

目前它的精确时间控制仍主要集中在对白，官方也明确表示未来会继续扩展到 SFX、环境声和音乐，因此它更像是下一阶段生成音频的发展方向。

3. 对游戏开发意味着什么？

我认为 AIGC 最先改变的不会是最终混音，而是游戏声音资产生产。

传统流程：

需求 ↓ 搜索音效库 / 录音 ↓ 剪辑 ↓ 制作多个变体 ↓ 导入 Wwise / FMOD

新的流程可能变成：

游戏设计需求 ↓ AI 快速生成声音 ↓ Audio-to-Audio / Inpainting 修改 ↓ 人工筛选和声音设计 ↓ Wwise / FMOD 构成交互声音系统

尤其适合：

游戏原型、独立游戏、大量 NPC / 场景音效、攻击与技能变体、环境 Ambience、Foley、UI 音效。

4. AI 还没有解决的问题

目前 Text-to-SFX 最大的问题已经不是“能不能生成”，而是能不能稳定控制。

游戏要求声音必须可重复、可预测，并与程序逻辑配合。

例如一个脚步系统还需要判断：

地面材质、角色速度、左右脚、距离、空间混响、随机变化。

AI 可以制作脚步素材，但不能自动代替整个游戏音频系统。

因此 Wwise、FMOD 和游戏引擎中的逻辑仍然非常重要。

5. 结论

2024 年左右，Text-to-SFX 最吸引人的地方还是：

“输入文字居然能生成声音。”

到了 2026 年，更值得关注的问题已经变成：

“怎样让 AI 成为真正的声音设计工具？”

目前的发展趋势十分明显：

Text-to-SFX → Voice / Audio 引导生成 → Audio Editing → DAW 集成 → Scene-level Audio Generation

因此我认为，AIGC 不会简单取代游戏音效设计师。

它更可能改变音效设计师获取和制作声音素材的方式。

未来游戏音频设计的核心能力，也可能从单纯的“寻找和剪辑素材”，进一步变成：

设计声音 + 控制生成模型 + 构建交互声音系统。

参考资料

以下均为官方资料，访问与复核日期为 2026 年 11 月 12 日。

1. ElevenLabs，Sound effects 文档：
https://elevenlabs.io/docs/overview/capabilities/sound-effects

2. ElevenLabs，Introducing SFX v3 in ElevenCreative and ElevenAPI，2026-10-29：https://elevenlabs.io/blog/sound-effects-v3

3. Adobe，Generate sound effects using text prompts：https://helpx.adobe.com/firefly/web/work-with-audio-and-video/work-with-audio/text-to-sound-effects.html

4. Adobe，Generate sound effects using your voice as a guide：https://helpx.adobe.com/firefly/mobile/work-with-audio-and-video/work-with-audio/generate-sound-effects-using-your-voice-as-a-guide.html

5. Stability AI，Stable Audio 3 Prompt Guide： https://kb.stability.ai/knowledge-base/stable-audio-3-prompt-guide

6. Stability AI，Stable Audio DAW Plugin Guide： https://kb.stability.ai/knowledge-base/stable-audio-daw-plugin-guide

7. ByteDance Seed，From Speech to Audio Creation Introducing the Seed Audio 1.0 Audio Creation Model，2026-07-20： https://seed.bytedance.com/en/blog/from-speech-to-audio-creation-introducing-the-seed-audio-1-0-audio-creation-model

6. 与本次期末作品的联系

镇守之森的制作采用“AI 生成部分音效 + CC0 自然录音 + 后期处理 + Wwise 调度”的混合流程。风声、草丛和铃音通过 Stable Audio 官方网页生成，流水、虫鸣和鸟鸣来自 CC0 素材。第一次生成的风声听起来接近雨声，重新生成后仍需要选段、滤波和降低音量，才接近哀怨轻柔的目标。

这次实践让我更明确地看到，模型输出只是素材起点。最终能否用于游戏，还要看循环是否平顺、文件规格是否统一、随机声是否过密，以及声源位置变化后音量是否合理。工具版本较新并不等于无需筛选，也不能代替真实试听和中间件测试。完整作品、来源和测试记录：

https://github.com/kirito-firefly/SHU-Game-Audio-Practice/tree/main/Final

三、期末作品

1. 期末作品 GitHub 链接

https://github.com/kirito-firefly/SHU-Game-Audio-Practice/tree/main/Final

完整作品说明：https://github.com/kirito-firefly/SHU-Game-Audio-Practice/blob/main/Final/DESIGN.md

九十秒试听：https://github.com/kirito-firefly/SHU-Game-Audio-Practice/blob/main/Final/Preview/ChinjuForest_90s.wav

2. 作品说明（可包含截图等内容）

1. 场景选择与听觉目标

我选择《原神》稻妻的镇守之森作为这次环境声设计的对象。吸引我的是古树、石阶、鸟居、石灯笼和蓝色微光组合出的幽静感。声音希望偏向哀怨、轻柔和孤寂，让人感到森林很深，同时保留少量自然活动。



图 1　镇守之森场景参考，本人在《原神》中拍摄。鸟居、石阶、古树与蓝色花丛构成本次声音设计的视觉依据。

我的思路是先建立安静的环境底色，再加入少量前景细节。近处的草丛要短、细，鸟鸣分散在中景，流水和铃音退到远处。风声不做成强风，虫鸣不做成热闹的夏夜，鸟鸣也不密集出现。铃音只保留单次敲击与尾响，不承担旋律音乐的作用。瀑布和铃音是画面外的创意声源，不能从这张参考图直接确认。

声音层	正式文件与数量	时长	设计作用与播放方式
微风与树叶	AMB_WindLeaves_Loop.wav 1 条	60 秒	全局循环，轻柔的空气流动与少量叶声
远处流水	AMB_Waterfall_Loop.wav 1 条	60 秒	循环、3D 衰减，提供远景纵深
细微虫鸣	AMB_Insects_Loop.wav 1 条	60 秒	全局循环，衬托寂静
零星鸟鸣	SFX_Bird_01/02/03.wav 3 条	各 4 秒	随机播放，少量自然生机
草丛窸窣	SFX_Rustle_01/02/03.wav 3 条	各 4 秒	随机播放，近处活动细节
日式铃音	SFX_Bell_01/02/03.wav 3 条	各 10 秒	低密度随机播放，表现清冷与孤寂

2. 确定设计对象与收集素材的过程

这次采用 AI 生成与现成自然录音混合制作。AI 用于比较容易描述动作和情绪的风声、草丛和铃音；真实录音用于补充流水、虫鸣和鸟鸣的自然细节。没有使用游戏原声，也没有进行野外实录。AI 辅助了提示词、处理脚本、工程配置与整理，我主要确定声音方向、判断样音是否合适，并根据试听提出修改要求。

生成工具使用 Stable Audio 官方网页，实际模型标签为 default (Stable Audio 3)。采用标准质量，随机种子为 random，steps 与 cfg_scale 为 auto；页面未提供实际种子数值。草丛生成 4 秒 × 3，铃音生成 10 秒 × 3，轻柔风声生成 30 秒 × 2。连同弃用的首批风声，四批共消耗 48 个免费积分，支出为 0 元。实际请求完整保存在 Evidence/generation_requests.json。

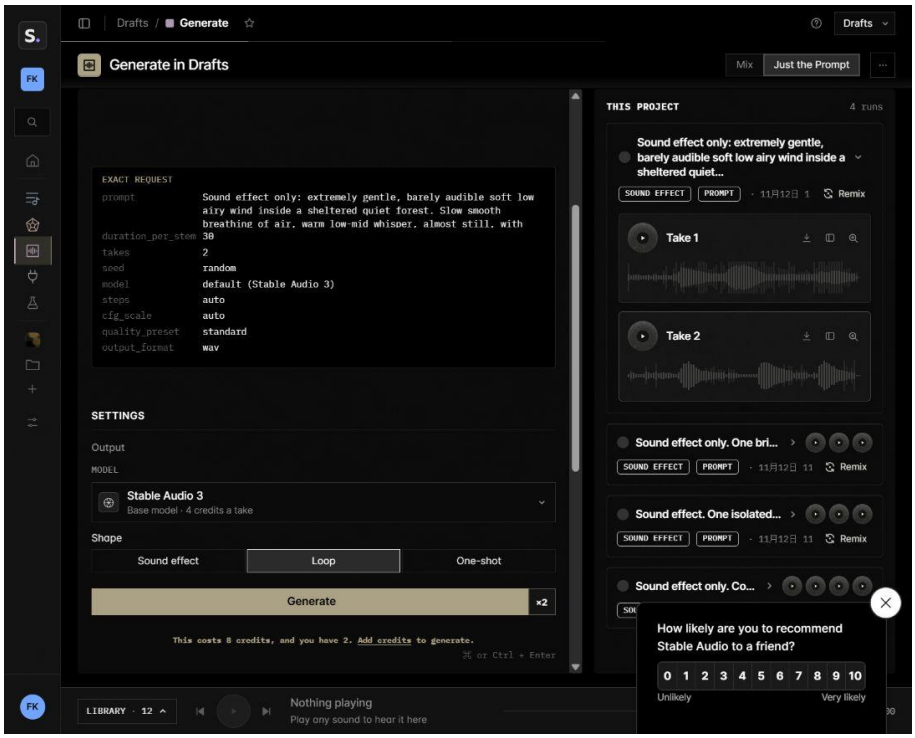


图 2　Stable Audio 实际生成页面。截图用于记录工具与生成结果；采用的请求以 JSON 中的完整英文提示词为准。

风声提示词强调 sheltered quiet forest、barely audible、almost still、occasional faint leaf rub，并排除雨、水流、密集沙沙声、音乐和人声。草丛强调小动物轻擦草叶的一次短促动作；铃音强调旧日式铜铃的单次轻敲、柔和起音与较长的清冷余响，避免连续叮当或旋律。

自然录音来自两个标注 CC0 的页面。Thimras 的 Park ambiences 提供河流与鸟鸣，录于澳大利亚阿德莱德；Wolfgang_ 的 Crickets Ambient Noise - loopable 提供虫鸣。三个鸟鸣文件是不同录音选段，不能据此声称是三种鸟。河流录音经处理后作为远处瀑布的听觉替代，也不应描述成现场瀑布实录。

- 公园河流与鸟鸣：https://opengameart.org/content/park-ambiences
- 虫鸣：https://opengameart.org/content/crickets-ambient-noise-loopable
- CC0 说明：https://creativecommons.org/publicdomain/zero/1.0/
- Stable Audio 使用授权：https://kb.stability.ai/knowledge-base/stable-audio-commerical-and-usage-licensing

选用源文件保存在 Sources/，完整来源、下载入口、积分和格式记录见 SOURCES.md。AI 下载源实际为 44.1 kHz、16 bit 双声道，后续转换成课程要求的规格；升采样和增加位深不会凭空增加源录音的信息量。

3. 剪辑素材与导出资产的过程

处理时先选取相对干净的部分，再做去直流、高低通滤波、音量调整与淡化。河流选取原录音 30—100 秒，削弱低频隆隆声和高频近景感；鸟鸣选取彼此分离的三个 4 秒片段，削弱不需要的环境底噪。虫鸣保留较细的高频质感，但降低整体音量。三个环境层均制作成 60 秒，循环接缝使用 0.5 秒交叉淡化。

风声是修改最多的部分。首批听起来太杂，接近雨声，不符合安静森林的目标。重新生成后，我又提出“再轻一些、叶声更少”。最后采用第二批 Take 1 的 1—5.5 秒与 18.5—23.5 秒，以 0.25 秒交叉淡化连接，使用 45 Hz 高通、1100 Hz 低通，并将 RMS 上限降至 -33 dBFS。这样空气声更轻，叶声也不会一直抢注意力。

草丛保留短促摩擦的变化，处理起止点，避免突然截断。铃音保留金属起音和自然衰减，文件末尾做淡出，不把尾响硬切掉。滤波、选段与淡化由 FFmpeg 和 Python 辅助完成，参数保存在 Audio_Edit/edit_decisions.json。REAPER 7.85 用于实际载入媒体、六轨编排与最终混音导出，工程使用相对路径。

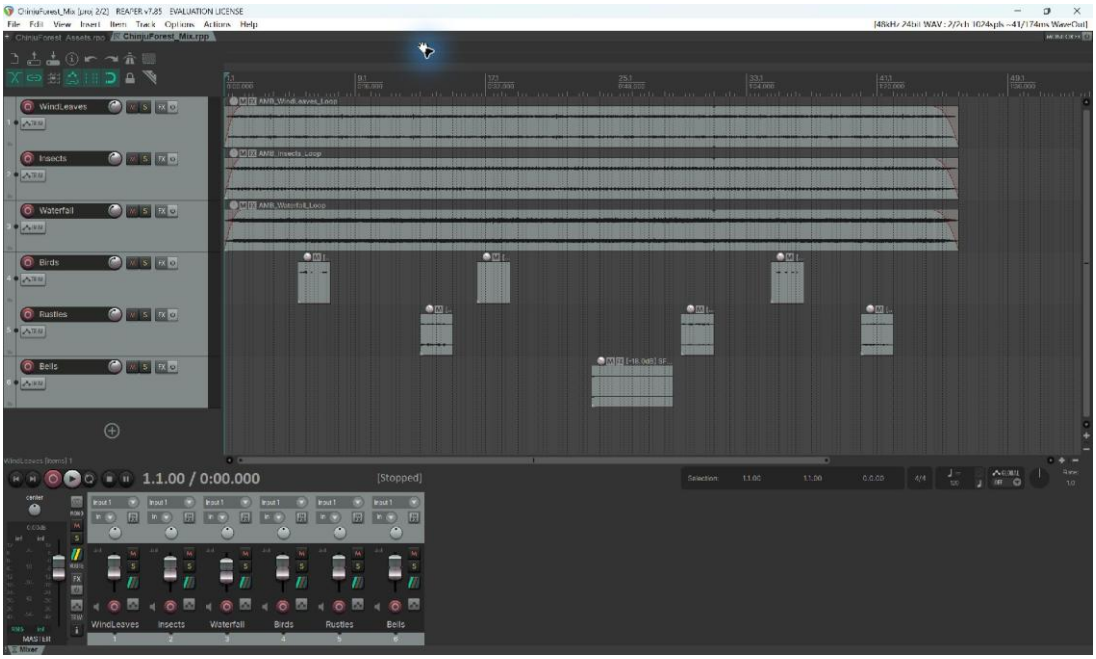


图 3　REAPER 中实际打开的六轨混音。三个环境底层持续铺设，鸟鸣、草丛和铃音间隔出现，整体使用淡入淡出。

正式资产共十二个，统一为 48 kHz、24 bit、双声道 PCM WAV。九十秒试听由 REAPER 原生导出：风、虫、水构成连续底层，鸟鸣安排在约 9、31、67 秒，草丛在 24、56、78 秒，铃音在 45 秒。它用于展示固定的声音编排；Wwise 内的随机播放另行测试。

自动检查确认十二个文件数量、时长、采样率、位深与声道满足要求，无削波和整段静音。九十秒混音峰值约 -9.77 dBFS，RMS 约 -28.81 dBFS，保留了一定余量。最终试听后，风声与叶声的轻重基本合适，整体方向确定为哀怨轻柔。十分钟引擎记录用于核对播放逻辑，不能代替人耳对循环接缝的逐次判断。

4. 使用 Wwise 的情况

工程使用课程指定的 Wwise 2025.1.11.9262，英文名称为 ChinjuForest_Ambience。在 ChinjuForest 总容器下建立 WindLeaves、Waterfall、Insects、Birds、Rustles、Bells 六层。三个持续层无限循环，分层控制音量；三组随机层各装入三个文件，采用 Random Container 的 Shuffle 连续模式，避免紧邻重复。

层	容器音量	播放结束后随机等待	首次延迟	最大预览距离
WindLeaves	-0.5 dB	持续循环	0	全局
Waterfall	-7 dB	持续循环	0	160 米
Insects	-3 dB	持续循环	0	全局
Birds	-12 dB	12—25 秒	8—15 秒	60 米
Rustles	-9 dB	18—35 秒	15—25 秒	15 米
Bells	-10 dB	45—80 秒	30—50 秒	120 米

这些等待时间从文件播完后计算。铃音十秒的文件还会占用播放时间，所以不会每隔四十五秒必定响一次。首次延迟错开，避免场景开始时三组声音同时出现。瀑布及三组随机声开启 3D 定位和衰减，最大距离的 0%、10%、50%、100% 分别设为 0、-3、-18、-60 dB。

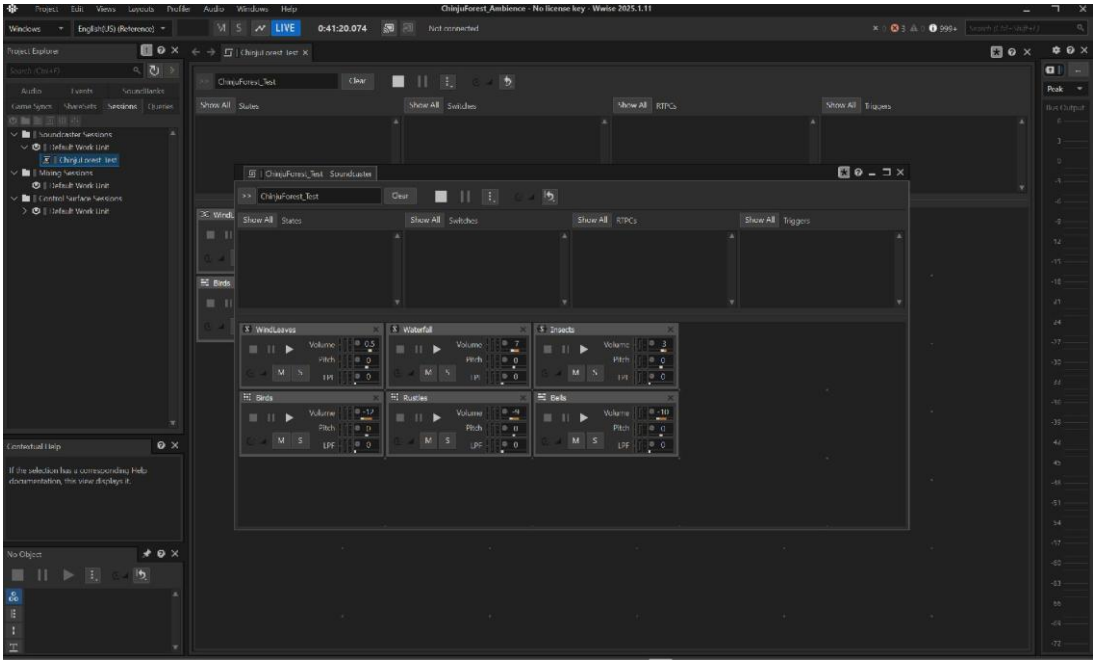


图 4　保存的 ChinjuForest_Test 会话，六个模块分别试听各层。工程另有总场景 Play_ChinjuForest、Stop_ChinjuForest 及六个分层 Play 事件。单独试听和总场景同时开启会叠加，需要避免重复播放。

通过官方 WAAPI 配置并记录参数，WwiseConsole 工程校验通过。完整场景在实际 Authoring 引擎中连续采样 604.92 秒，共 1194 次快照。每次快照均能观察到三个持续环境层的非虚拟声部，九个随机变体全部出现。估计的播放后等待为鸟鸣 12.275—24.474 秒、草丛 20.316—34.577 秒、铃音 60.015—76.386 秒，均符合设定范围。采样粒度约 0.5 秒，判断允许 0.8 秒误差，排除首个不完整间隔。初始延迟核对了保存配置，本次未测首个触发的精确时间。停止事件执行两秒后，场景 transport 停止且无残留声部。

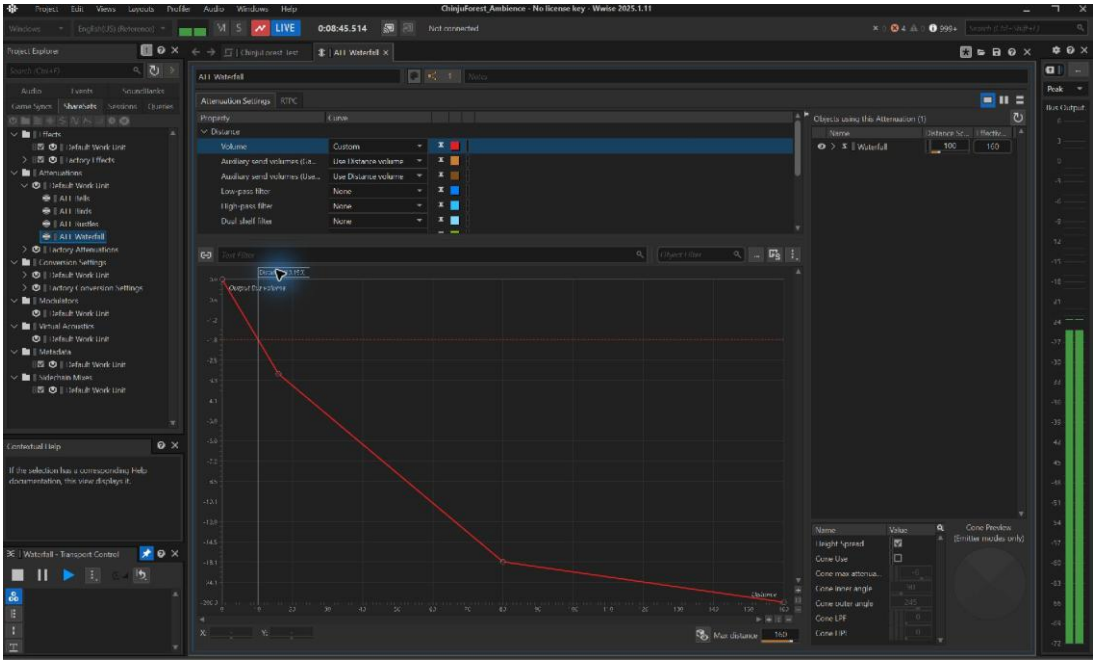


图 5　瀑布近距离预览，距离游标约 10 米。

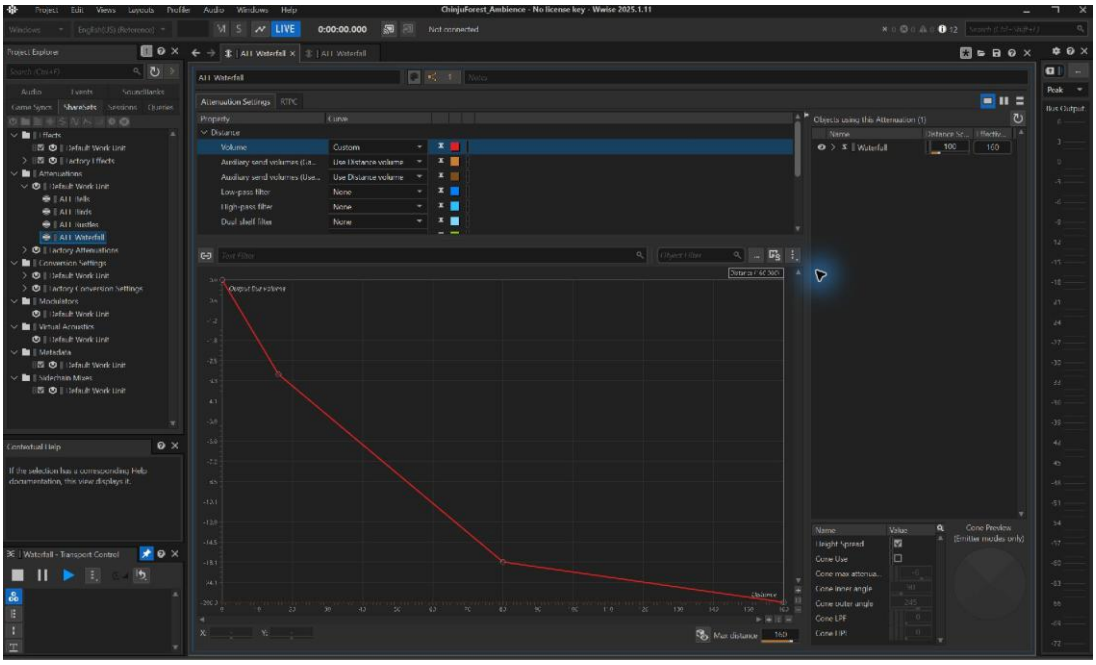


图 6　瀑布远距离预览，距离游标为 160 米。与近距离相比，输出电平明显降低。

另外注册独立声源与监听器，改变实际坐标并读取 Voice Inspector 的声部贡献。瀑布在 0、16、80、160 米的总增益约为 -7、-10、-25、-67 dB；鸟鸣、草丛和铃音在相同比例距离也分别得到 0、-3、-18、-60 dB 的相对衰减，数值误差小于 0.02 dB。界面预览与引擎数值相互核对，说明距离变化确实影响了音量。

5. 对作品的自我评价

我比较满意的是声音密度的控制。前两次风声并不理想，继续减少叶声、削弱高频和降低音量以后，才更接近我希望的哀怨轻柔感。铃音采用长等待，鸟鸣与草丛也留出空白，环境不会一直有声音事件抢占注意力。这次让我体会到，环境声设计需要主动做减法，素材多并不代表场景更有层次。

作品也有明确的限制。鸟鸣来自澳大利亚公园，适合本次氛围原型，但不能当作日本森林生态的准确复原。流水是河流录音加工，铃音是创意声源。当前完成的是 Wwise Authoring 内的原型，尚未接入游戏关卡；米制距离、方位和真实移动声源仍需在游戏中结合尺度继续调整。AI 对提示词的遵循也不稳定，生成后依然需要筛选、编辑和试听。

课程工程、声音来源和处理参数都已保留，便于复核。本次实践把平时调研中的生成式音频思路落实为可播放、可调整的资产与中间件工程，也让我理解了采样规格、循环、随机调度和空间衰减分别解决什么问题。

6. 完整提交入口

- 期末作品与工程：https://github.com/kirito-firefly/SHU-Game-Audio-Practice/tree/main/Final
- 九十秒试听：https://github.com/kirito-firefly/SHU-Game-Audio-Practice/blob/main/Final/Preview/ChinjuForest_90s.wav
- 平时调研：https://github.com/kirito-firefly/SHU-Game-Audio-Practice/blob/main/Task/AIGC_Game_Audio_Research.md
- 素材来源：https://github.com/kirito-firefly/SHU-Game-Audio-Practice/blob/main/Final/SOURCES.md
- 测试记录：https://github.com/kirito-firefly/SHU-Game-Audio-Practice/blob/main/Final/TESTING.md

打开工程时保留 Final 文件夹结构。Wwise 从 Events 试听总场景，并用 Stop_ChinjuForest 停止；Soundcaster 六个模块用于分层检查。REAPER 的 Mix 工程用于固定编排试听，Assets 工程用于十二个正式资产的对照检查。

注：期末报告请以 PDF 格式上传至智慧实习实训管理平台“过程文档”中，11 月 15 日截止